Лабораторная работа №1: Динамическое программирование на подмножествах. Сегментация текста с весовой функцией.

Слёлин А.В. ПМИ-11МО

2 марта, 2026 г.

Постановка задачи

Пусть дано:

- Строка S длины n (индексация символов от 0 до $n-1$)
- Словарь D, содержащий слова с их весами: $D = \{w_1 : \text{weight}(w_1), w_2 : \text{weight}(w_2), \ldots, w_m : \text{weight}(w_m)\}$
- Максимальная длина слова в словаре $L = \max\limits_{w \in D} |w|$

Требуется найти разбиение строки S на последовательность слов $w_1, w_2, \ldots, w_k$ такую, что:

1. $w_1 + w_2 + \ldots + w_k = S$ (конкатенация слов равна исходной строке)
2. Каждое $w_i \in D$
3. Сумма весов $\sum\limits_{i=1}^{k} \text{weight}(w_i)$ максимальна

Теория

Динамическое программирование на подмножествах

В классическом понимании «ДП на подмножествах» — это перебор всех подмножеств элементов с помощью битовых масок (например, задача коммивояжера). В данном контексте мы имеем дело с линейной структурой (строка), но суть та же: состояние определяется множеством уже обработанных символов, которое задается индексом. Каждый следующий шаг — это выбор слова, «покрывающего» непрерывный отрезок строки.

Определение состояния

Введем массив dp размером $n+1$, где:

$$dp[i] = \begin{array}{l}\text{максимальная сумма весов для префикса строки } S[0..i-1] \\ \text{(первые } i \text{ символов)}\end{array}$$

Например:

- $dp[0]$ — для пустой строки
- $dp[1]$ — для первого символа $S[0]$
- $dp[2]$ — для первых двух символов $S[0..1]$
- $dp[n]$ — для всей строки S (это и будет ответ)

База динамики

$$dp[0] = 0$$

Пустая строка имеет вес 0. Все остальные $dp[i]$ изначально равны $-\infty$ (в реализации используем $Long.MIN_VALUE$), что означает "невозможно составить такой префикс из слов словаря".

Рекуррентное соотношение

Для каждого i от 1 до n:

$$dp[i] = \max_{\substack{0 \leqslant j < i \\ i-j \leqslant L \\ S[j:i] \in D}} \big(dp[j] + \text{weight}(S[j:i])\big)$$

где:

- j — индекс начала последнего слова
- $S[j:i]$ — подстрока с j по i (не включая i), длиной $i - j$
- $\text{weight}(S[j:i])$ — вес слова из словаря
- $i - j \leqslant L$ — ограничение на длину слова

Оптимизация с использованием ограничения длины

Так как максимальная длина слова в словаре равна L, мы можем рассматривать только те j, для которых $i - j \leqslant L$. Это означает, что начало слова может находиться не далее чем на L символов от текущей позиции:

$$j \in [\max(0, i - L), i - 1]$$

Это снижает сложность алгоритма с $O(n^2)$ до $O(n \cdot L)$, где L — максимальная длина слова в словаре.

Восстановление ответа

Для восстановления разбиения используем два вспомогательных массива:

- $prev[i]$ — индекс начала последнего слова, которое привело к оптимальному значению $dp[i]$
- $word[i]$ — само слово, которое было использовано для перехода в состояние i

Восстановление происходит от конца к началу:

1. Начинаем с $i = n$
2. Добавляем $word[i]$ в начало результирующего списка
3. Переходим к $i = prev[i]$
4. Повторяем, пока $i > 0$

Сложность алгоритма

- Время работы: $O(n \cdot L)$, где n — длина строки, L — максимальная длина слова в словаре
- Дополнительная память: $O(n)$ для хранения массивов dp, $prev$ и $word$

Обработка невозможного разбиения

Если $dp[n] = -\infty$ (в реализации $Long.MIN_VALUE$), то разбиение строки на слова из словаря невозможно.

Код

Пример кода на Java:

Листинг 1: Алгоритм

```java
package a.slelin.work.laba1;

import lombok.*;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Map;

@NoArgsConstructor(access = AccessLevel.PRIVATE)
public class Algorithm {

    @Getter
    @Setter
    @AllArgsConstructor
    public static class Result {

        private Long totalWeight;

        private List<String> words;

        public static Result unresolved() {
```

```
23              return new Result(-1L, null);
24          }
25
26          public boolean isUnresolved() {
27              return this.totalWeight == -1L && this.words == null
                    ;
28          }
29
30      }
31
32      public static Result solve(String S, Map<String, Long>
            dictionary, Long L) {
33
34          if (S == null || dictionary == null || L == null) {
35              throw new IllegalArgumentException("Значения для реш
                    ения алгоритма не должны быть null!");
36          }
37
38          int n = S.length();
39
40          /*
41           * Создаем базовый массив для записи максимального набор
                 а весов для слова длины i.
42           * [a, b, a, b, c]
43           * [0, -1, -1, -1, -1, -1]
44           * В последней ячейке будет находиться ответ, причём есл
                 и он -1, то значит невозможно
45           * составить слово по словарю.
46           * */
47          long[] dp = new long[n + 1];
48          Arrays.fill(dp, Long.MIN_VALUE);
49          dp[0] = 0;
50
51          // Для хранения информации о разбиении
52          int[] prevIndex = new int[n + 1];
53          String[] usedWord = new String[n + 1];
54          Arrays.fill(prevIndex, -1);
55
56          // Для оптимизации создания подстрок используем char[]
57          char[] chars = S.toCharArray();
58
59          // Будем идти по каждой букве строки S и заполнять масси
                в dp
60          for (int i = 1; i <= n; i++) {
61
62              // Будем перебирать подстроки, которые длины меньше,
                     чем L, от текущей буквы.
```

```
            long start = Math.max(0, i - L);

            // Используем StringBuilder для эффективного создания подстрок
            StringBuilder sb = new StringBuilder();

            // Перебираем возможные начала слов от i-1 до start
            for (int j = i - 1; j >= start; j--) {

                // Вставляем символ в начало StringBuilder
                sb.insert(0, chars[j]);
                String candidate = sb.toString();

                /*
                 * Проверяем есть ли такая подстрока в словаре.
                 * Если нет, то пропускаем её, если же есть, то необходимо посчитать
                 * сумму веса этой подстроки с её уже оптимальным значением веса.
                 * Далее следует выбрать максимальную и запомнить её.
                 * [a, b, a, b]
                 * [1, 2, 3, ?]
                 * a:2,ba:2
                 * Для aba - так как такой строки нет, то пропускаем.
                 * Для ba - weight = 2 + 2 = 4
                 * Для a - weight = 3 + 2 = 5
                 * Значит максимальное на этом этапе 5
                 * [1, 2, 3, 5] и запоминаем строку "a".
                 * */
                Long weight = dictionary.get(candidate);
                if (weight != null && dp[j] != Long.MIN_VALUE) {
                    long newWeight = dp[j] + weight;

                    if (newWeight > dp[i]) {
                        dp[i] = newWeight;
                        prevIndex[i] = j;
                        usedWord[i] = candidate;
                    }
                }
            }
        }

        // Если последнее значение в массиве равно -1, то невозможно составить слово по словарю.
        if (dp[n] == Long.MIN_VALUE) {
```

```
            return Result.unresolved();
        }

        // Восстанавливаем разбиение
        List<String> words = new ArrayList<>();
        int current = n;

        while (current > 0) {
            words.add(usedWord[current]);
            current = prevIndex[current];
        }
        words = words.reversed();

        return new Result(dp[n], words);
    }
}
```

Тесты

Проведем базовое тестирование алгоритма с помощью 3 методов.

Листинг 2: Базовые тесты

```
package a.slelin.work.laba1.test;

import a.slelin.work.laba1.Algorithm;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.Timeout;

import java.util.List;
import java.util.Map;
import java.util.concurrent.TimeUnit;

import static org.junit.jupiter.api.Assertions.*;

@DisplayName("Базовый тест из условий")
public class SimpleTest {

    public static final Map<String, Long> map = Map.of(
            "ab", 2L,
            "abc", 5L,
            "bc", 3L,
            "ca", 4L,
            "a", 1L,
            "b", 1L,
            "c", 1L
```

```
25      );
26
27      @Test
28      @DisplayName("Тест со строкой \"abc\"")
29      public void testAbc() {
30          Algorithm.Result result = Algorithm.solve("abc", map, 3L
            );
31          assertNotNull(result);
32          assertNotNull(result.getTotalWeight());
33          assertEquals(5L, result.getTotalWeight());
34          assertNotNull(result.getWords());
35          assertEquals(List.of("abc"), result.getWords());
36      }
37
38      @Test
39      @DisplayName("Тест со строкой \"ababc\"")
40      public void testAbabc() {
41          Algorithm.Result result = Algorithm.solve("ababc", map,
            3L);
42          assertNotNull(result);
43          assertNotNull(result.getTotalWeight());
44          assertEquals(7L, result.getTotalWeight());
45          assertNotNull(result.getWords());
46          assertEquals(List.of("a", "b", "abc"), result.getWords()
            );
47      }
48
49      @Test
50      @DisplayName("Тест со строкой \"xyz\"")
51      public void testXyz() {
52          Algorithm.Result result = Algorithm.solve("xyz", map, 3L
            );
53          assertNotNull(result);
54          assertTrue(result.isUnresolved());
55          assertNotNull(result.getTotalWeight());
56          assertEquals(-1L, result.getTotalWeight());
57          assertNull(result.getWords());
58      }
59
60      @Test
61      @DisplayName("Тест со строкой \"a\"")
62      public void testSingleCharA() {
63          Algorithm.Result result = Algorithm.solve("a", map, 3L);
64          assertNotNull(result);
65          assertEquals(1L, result.getTotalWeight());
66          assertEquals(List.of("a"), result.getWords());
67      }
```

```java

    @Test
    @DisplayName("Тест со строкой \"ab\"")
    public void testExactMatchAb() {
        Algorithm.Result result = Algorithm.solve("ab", map, 3L);
        assertNotNull(result);
        assertEquals(2L, result.getTotalWeight());
        assertEquals(List.of("a", "b"), result.getWords());
    }

    @Test
    @DisplayName("Тест со строкой \"abcabc\"")
    public void testAbcabc() {
        Algorithm.Result result = Algorithm.solve("abcabc", map, 3L);
        assertNotNull(result);
        assertEquals(10L, result.getTotalWeight());
        assertEquals(List.of("abc", "abc"), result.getWords());
    }

    @Test
    @DisplayName("Тест со строкой \"abca\" - поиск оптимального разбиения")
    public void testAbca() {
        Algorithm.Result result = Algorithm.solve("abca", map, 3L);
        assertNotNull(result);
        assertEquals(6L, result.getTotalWeight());
        assertEquals(List.of("abc", "a"), result.getWords());
    }

    @Test
    @DisplayName("Тест со строкой \"cabc\"")
    public void testCabc() {
        Algorithm.Result result = Algorithm.solve("cabc", map, 3L);
        assertNotNull(result);
        assertEquals(7L, result.getTotalWeight());
        assertEquals(List.of("ca", "bc"), result.getWords());
    }

    @Test
    @DisplayName("Тест с максимальной длиной слова 2")
    public void testWithMaxWordLength2() {
        Algorithm.Result result = Algorithm.solve("abcabc", map, 2L);
```

```java
        assertNotNull(result);
        assertEquals(9L, result.getTotalWeight());
        assertEquals(List.of("a", "b", "ca", "bc"), result.getWords());
    }

    @Test
    @DisplayName("Тест со строкой \"ac\"")
    public void testAcNoDirectMatch() {
        Algorithm.Result result = Algorithm.solve("ac", map, 3L);
        assertNotNull(result);
        assertEquals(2L, result.getTotalWeight());
        assertEquals(List.of("a", "c"), result.getWords());
    }

    @Test
    @DisplayName("Тест со строкой \"bac\"")
    public void testBac() {
        Algorithm.Result result = Algorithm.solve("bac", map, 3L);
        assertNotNull(result);
        assertEquals(3L, result.getTotalWeight());
        assertEquals(List.of("b", "a", "c"), result.getWords());
    }

    @Test
    @DisplayName("Тест с пустой строкой")
    public void testEmptyString() {
        Algorithm.Result result = Algorithm.solve("", map, 3L);
        assertNotNull(result);
        assertEquals(0L, result.getTotalWeight());
        assertTrue(result.getWords().isEmpty());
    }

    @Test
    @DisplayName("Тест со строкой, которую невозможно разбить")
    public void testImpossibleString() {
        Algorithm.Result result = Algorithm.solve("abcd", map, 3L);
        assertNotNull(result);
        assertTrue(result.isUnresolved());
        assertEquals(-1L, result.getTotalWeight());
        assertNull(result.getWords());
    }

    @Test
```

```java
    @DisplayName("Тест с максимальной длиной слова 1")
    public void testWithMaxWordLength1() {
        Algorithm.Result result = Algorithm.solve("abc", map, 1L);
        assertNotNull(result);
        assertEquals(3L, result.getTotalWeight());
        assertEquals(List.of("a", "b", "c"), result.getWords());
    }

    @Test
    @DisplayName("Тест со строкой \"caab\"")
    public void testCaab() {
        Algorithm.Result result = Algorithm.solve("caab", map, 3L);
        assertNotNull(result);
        assertEquals(6L, result.getTotalWeight());
        assertEquals(6L, result.getTotalWeight());
    }

    @Test
    @DisplayName("Тест с очень большой максимальной длиной слова")
    public void testWithVeryLargeMaxLength() {
        Algorithm.Result result = Algorithm.solve("abc", map, 100L);
        assertNotNull(result);
        assertEquals(5L, result.getTotalWeight());
        assertEquals(List.of("abc"), result.getWords());
    }

    @Test
    @DisplayName("Тест с большой строкой")
    @Timeout(value = 5, unit = TimeUnit.SECONDS)
    public void testRandomValidString() {
        Algorithm.Result result = Algorithm.solve("abc".repeat(3333) + "a", map, 3L);
        assertNotNull(result);
        assertEquals(16666L, result.getTotalWeight());
        assertNotNull(result.getWords());
        assertEquals(3334, result.getWords().size());
    }
}
```

Все тесты успешно пройдены!

Выводы

Мы написали алгоритм для решения лабораторной работы №1 по теме: "Динамическое программирование на подмножествах. Сегментация текста с весовой функцией."варианта №3.

Освоили метод динамического программирования на подмножествах для решения задачи сегментации последовательности. Изучили способы модификации классического алгоритма «Word Break» для работы с весами слов, с ограничениями на структуру словаря и с нечеткими совпадениями. Научились восстанавливать оптимальное разбиение по нескольким критериям.

Все поставленные перед нами задачи были выполнены!